Operators

🔘 ARITHMETIC OPERATORS (+ * / % **)

1. Predict the output:

```
console.log(10 + 20);
```

2. Predict the output:

```
console.log('10' + 20);
```

3. Predict the output:

```
console.log('10' - 2);
```

4. Predict the output:

```
console.log(10 * '2');
```

5. Predict the output:

```
console.log(10 / 0);
```

6. Predict the output:

```
console.log(0 / 10);
```

7. Predict the output:
8. Predict the output:

```
console.log(2 ** 3 ** 2);
```

9. Why does `+` behave differently from other arithmetic operators?

🟡 ASSIGNMENT OPERATORS (= += *= /= %=)

1. Predict the output:

```
console.log(5 % 2);
```

```
let a = 10;
a += 5;
console.log(a);
```

2. Predict the output:

```
let b = 10;
b *= 2 + 3;
console.log(b);
```

3. Predict the output:

```
let x = 5;
x = x++ + ++x;
console.log(x);
```

4. Predict the output:

```
let y = 10;
y += y -= 5;
console.log(y);
```

5. Why are compound assignment operators dangerous in complex expressions?

COMPARISON OPERATORS (== === != !== > < >= <=)

1. Predict the output:

```
console.log(5 == '5');
```

2. Predict the output:

```
console.log(5 === '5');
```

3. Predict the output:

```
console.log(null == undefined);
```

4. Predict the output:

```
console.log(null === undefined);
```

5. Predict the output:

```
console.log([] == false);
```

6. Predict the output:

```
console.log([] === false);
```

7. Predict the output:

```
console.log('' == 0);
```

8. Why is `===` recommended over `==`?

LOGICAL OPERATORS (&& || !)

1. Predict the output:

```
console.log(true && false);
```

2. Predict the output:

```
console.log(false || 'JS');
```

3. Predict the output:

```
console.log('Hello' && 0);
```

4. Predict the output:

```
console.log(null || undefined || 'OK');
```

5. Predict the output:

```
console.log(!'');
```

6. Explain **short-circuiting** with a real example.

UNARY OPERATORS (++ -- typeof delete)

1. Predict the output:

```
let a = 5;
console.log(a++);
console.log(++a);
```

2. Predict the output:

```
console.log(typeof NaN);
```

3. Predict the output:

```
console.log(typeof typeof 10);
```

4. Predict the output:

```
let obj = { x: 10 };
delete obj.x;
console.log(obj.x);
```

5. Why does `typeof null` return `object`?

TERNARY OPERATOR (?:)

1. Predict the output:

```
let age = 18;
console.log(age >= 18 ? 'Adult' : 'Minor');
```

2. Predict the output:

```
let result = 0 ? 'Yes' : 'No';
console.log(result);
```

3. Predict the output:

```
let a = 10;
let b = 20;
let max = a > b ? a : b;
console.log(max);
```

4. Rewrite this using ternary operator:

```
if (x > 0) {
  y = 1;
} else {
  y = -1;
}
```

5. Why is excessive ternary nesting considered bad practice?

🌑 REAL-WORLD & INTERVIEW TRAPS

1. Predict the output:

```
console.log(1 && 2 && 3);
```

2. Predict the output:

```
console.log(0 || null || undefined || 'JS');
```

3. Predict the output:

```
console.log(!!'JavaScript');
```

4. Why does `[] + {}` produce a strange result?

5. Predict the output:

```
console.log(true + true + false);
```

6. Why is comparing objects using `==` or `===` unreliable?

🤔 CHALLENGE QUESTIONS (ADVANCED THINKING)

1. Predict the output:

```
console.log(10 > 5 > 1);
```

2. Predict the output:

```
console.log('5' > 3);
```

3. Predict the output:

```
console.log(null > 0);
console.log(null == 0);
console.log(null >= 0);
```

4. Explain **operator precedence** with an example.

5. Why does JavaScript allow implicit type coercion in operators?

6. Which operator causes the **maximum bugs in JavaScript** and why?

✅ INSTRUCTIONS

- Solve all questions without executing code • Write reasoning for each answer
- After completion, say:

🌐 **"Give me detailed answers for Operators questions"**

I will then explain: - Step-by-step execution - Type coercion rules - Memory & evaluation order - Interviewsafe explanations